

CSS Avanzado

Contenidos

Trucos de CSS

Responsive Design

Cascada CSS

Máscaras y recortes CSS

Lógica CSS

Inspirado en el bootcamp de
Manz.dev todos los créditos a el.

Colores derivados

- Con `color-mix()` puedes mezclar varios colores (uno puede ser `transparent`).
- Con `from` puedes crear colores derivados (y no sólo `rgb()` → `hsl()`, `oklch()`, `oklab()`...)

```
.element {  
  --color: #ff0000;    /* Rojo */  
  
  /* Mezcla de colores */  
  --soft-color: color-mix(in srgb, var(--color), white 80%);  
  --dark-color: color-mix(in srgb, var(--color), white 20%);  
  
  /* Colores relativos */  
  --same-color: rgb(from var(--color) r g b);    /* Mismo color: rgb(255 0 0) */  
  --pink-color: rgb(from var(--color) r g 255);  /* Color rosa: rgb(255 0 255) */  
}
```

Utilidades para imágenes

- Uso de fallbacks con **image-set** (si soporta, no descarga el resto)

```
.element {  
  background: image-set(  
    url("image1.avif") type("image/avif"),  
    url("image1.webp") type("image/webp"),  
    url("image1.jpg") type("image/jpeg")  
  );  
}
```

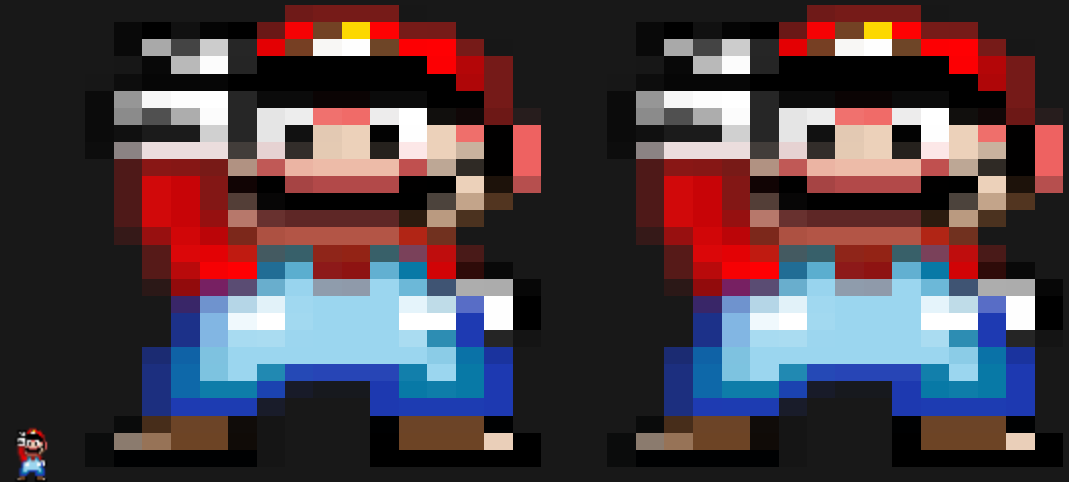
```
  
  
<style>  
img {  
  width: 256px; /* Cambiar a 128px */  
  height: 256px;  
  /* object-fit: cover; /* cover / contain */  
  /* object-position: top right; */  
  /* object-view-box: inset(15%); */  
  /* &:hover { object-view-box: inset(0); } */  
}  
</style>
```

Renderización de imágenes

- Imagen original tamaño 18x18
- Imagen redimensionada 256x256

```


<style>
  img {
    width: 256px;
    height: 256px;
    image-rendering: pixelated;
  }
</style>
```



Filtros y modos de fusión

- `filter`: `contrast(200%)`
- `background-color`: `blue`
- `background-blend-mode`: `darken`

```
img {  
  background-color: red;  
  background-image: url("duck.png");  
  filter: grayscale(100%);  
}
```

Responsive

Media Queries

Media Features

Preferencias de usuario

Container Queries



Filosofía del responsive

- Elige estrategia: **Mobile-first** / **Desktop-first**, Olvídate del «pixel perfect»
- Intenta reducir contenido en media queries (si puedes usar variables CSS, mejor)
- Establece tus breakpoints y crea las **media queries modernas** que necesites

```
/* Estilos comunes que no cambian de desktop a mobile */
.element {
  background: indigo;
}

/* Casos particulares: cambios de direccionalidad, paddings, etc... */
/* (Bloques @media siempre al final, para aprovechar herencia) */
@media ... {
  /* ... */
}
```

Filosofía del responsive

- Elige estrategia: **Mobile-first** / **Desktop-first**, Olvídate del «pixel perfect»
- Intenta reducir contenido en media queries (si puedes usar variables CSS, mejor)
- Establece tus breakpoints y crea las **media queries modernas** que necesites

```
/* Estilos aplicados a pantallas entre 640px y 1024px */  
  
@media screen and (min-width: 640px) and (max-width: 1024px) { }      /* ❌ LEGACY */  
  
@media (width >= 640px) and (width <= 1024px) { }                      /* ✅ MEJOR */  
  
@media (640px <= width <= 1024px) { }                                  /* ✅✅ AÚN MEJOR */
```

```
.element {  
  --pad: 2rem;  
  background: indigo;  
  padding: var(--pad);  
  display: flex;  
  
  @media (width <= 1024px) {  
    --pad: 0.5rem;  
    flex-direction: column;  
  }  
}
```

No sólo de width vive el dev

- Puedes usar height, aspect-ratio, orientation u otros
- Hay muchos [media features](#)
- También hay @media basados en las preferencias de usuario

```
@media (aspect-ratio <= 6/2) { }          /* 6/2 === 3/1 === 3 */
@media (orientation: landscape) { }      /* Landscape (apaisado) o portrait (retrato) */
@media (scripting: none) { }             /* El usuario no tiene Javascript */
@media (any-hover: none) { }             /* El usuario no tiene cursores (ratón) */
@media (any-pointer: none) { }           /* Usuario sin táctil (coarse) ni ratón preciso (fine) */
@media print { }                         /* Se aplica sólo cuando se imprime (físico o PDF) */
```

```
@media (prefers-reduced-motion: reduce) { } /* Usuario prefiere sin animaciones */
@media (prefers-reduced-transparency: reduce) { } /* Usuario prefiere sin transparencias */
@media (prefers-reduced-data: reduce) { } /* ⚠ Usuario prefiere ahorrar datos */
@media (prefers-color-scheme: dark) { } /* Usuario prefiere modo oscuro (light=claro) */

/* Forma alternativa (compacta y directa) */
body {
  background: light-dark(white, black);
  color: light-dark(black, white);
}
```

Container queries

```
.container {  
  background: grey;  
  display: flex;  
  gap: 1rem;  
  container: parent / inline-size; /* ← Establecemos padre */  
  width: 600px; /* ← Cambiar a 300px */  
  
  .item {  
    background: blue;  
    width: 200px;  
    height: 200px;  
    @container parent (width <= 500px) { background: red } /* ← Container query */  
  }  
}
```


Cascada CSS

Especificidad CSS

Capas de cascada @layer

:where() vs !important

Alcance con @scope

Shadow DOM declarativo



Especificidad CSS

- CSS tiene varias reglas → Cascada
- NO FUNCIONAN como la programación
- Hay que entenderlas (no memorizarlas)

Reglas

- Herencia: Quien tiene preferencia
- Estructura: Como se lee el CSS
- Especificidad: Resolver conflictos
- Alcance: Hasta donde se aplica el CSS



Herencia

- Propiedades heredables: `color`, `font-family`, `font-size`, etc...
- Valores de herencia: `inherit`, `initial` o `unset` (`inherit/initial` según caso)
- Resets si quieres algo ya hecho

```
<div class="parent">
Hello from parent!
<p class="child">Hello from child!</p>
</div>

<style>
.parent {
background: black;
color: gold;
}
</style>
```

```
.parent {
border: 2px solid deeppink;

.child {
border: inherit;
}
}
```

Estructura (tradicional)

- Fuente: [-] **<link>** → **<style>** → **style=""** [+]
- Orden (mismo selector): herencia + **!important**
- Selector: Especificidad (A,B,C) → Calculadoras: [keegan](#) / [wallace](#)

```
<div id="element" class="text">
  Texto del elemento
</div>

<style>
div { background-color: red; }
#element { background-color: steelblue; }
.text { background-color: green; }
</style>
```

- El selector div tiene especificidad...
- El selector #element tiene especificidad...
- El selector .text tiene especificidad...

Estructura (moderna)

- Estilos globales ↔ Estilos locales (Shadow DOM)
- Evita !important a favor de :where() (especificidad 0)
- Usa @layer (capas, estilo «Photoshop», de especificidad)

```
<div class="text">           <!-- CSS inyectado -->
Texto del elemento
</div>

<style>
:where(.text) { background-color: indigo }
.text { background-color: deeppink }
</style>
```

```
@layer base, theme;

.primary {
  @layer theme {
    background: indigo;
    color: white;
  }
  @layer base {
    background: grey;
    width: 250px;
    height: 200px;
    padding: 1rem;
  }
}
```

Alcance (scope)

- Nomenclatura BEM, ahora: **@scope**

```
<!-- Estilos acotados al div padre -->
<div class="parent">

  <style>
    @scope {
      h1 { color: purple }
    }
  </style>
  <h1>Hola a todos</h1>

</div>
<h1>Texto fuera</h1>
```

```
/* Nomenclatura BEM */
.parent__element--modifier {
  /* ... */
}
/* A partir de <div class="parent"> */
@scope (.parent) {
  div { color: red }
}
/* De div.parent a div.child */
@scope (.parent) to (.child) {
  div { color: blue }
}
```

Máscaras y recortes

Formas

Recortes (clip-path)

Shape (SVG-in-CSS)

Máscaras



Formas CSS (Aplicadas con clip-path)

- Sin recorte
- Rectángulos: `rect()`, `inset()` e `xywh()`
- Círculos y elipses: `circle()` y `ellipse()`
- Trayectos: `polygon()`, `path()` y `shape()`

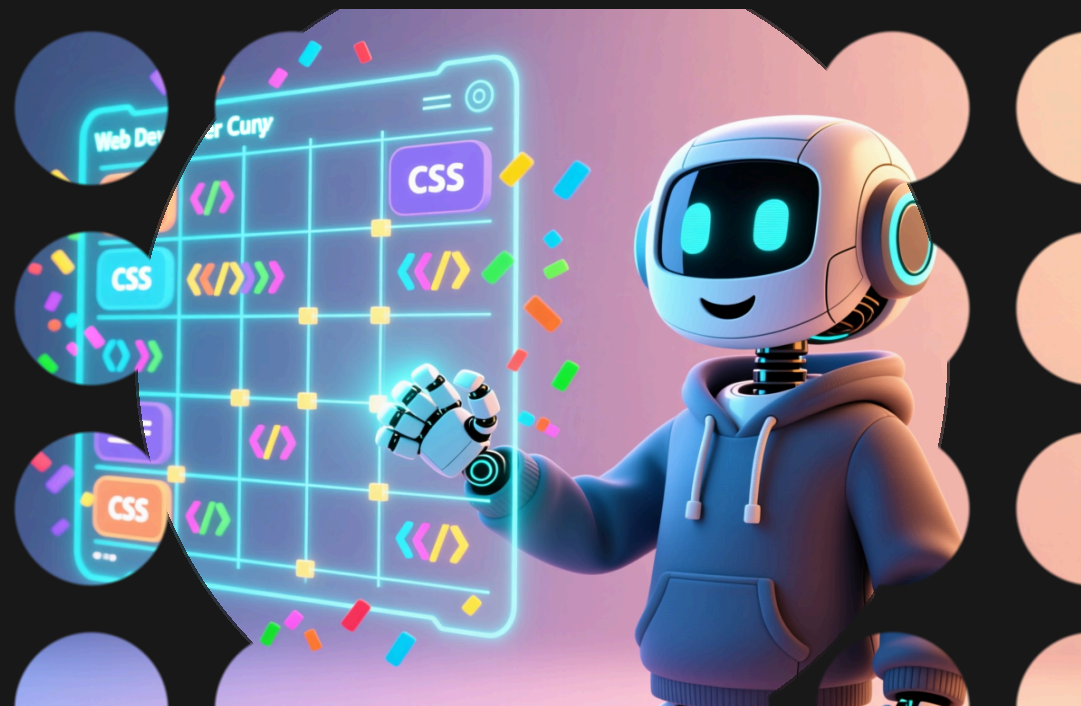
`clip-path: none`



Máscaras

- Tenemos una imagen y aplicamos máscara compuesta
- Un gradiente repetido y otro sin repetir

```
.element {  
  mask-image:  
    radial-gradient(circle 50px at 50px 50px,  
      black 75%, transparent 78%),  
    radial-gradient(circle 400px,  
      black 50%, transparent 50.5%);  
  mask-repeat: repeat, no-repeat;  
  mask-size: 100px 100px, cover;  
  mask-composite: add;  
}
```



Lógica CSS

¿CSS es programación?

if en CSS

function en CSS

Tipos de datos en CSS



Condicionales en CSS

- `@media (...)` según features del dispositivo
- `@media (prefers-*)` según preferencias de usuario
- `@support (...)` según soporte/compatibilidad
- `@container (...)` según contenedor padre
- `var(..., fallback)` según si existe variable
- `if(...; else: ...)` según valor de variable

```
@media (width <= 800px) { /* ... */ }
@media (prefers-color-scheme: dark) { /* ... */ }
@supports not (object-view-box: none) { /* ... */ }
@container name (width <= 800px) { /* ... */ }
.element { background: var(--color, grey) }
.element {
  --name: "ManzDev";
  background: if(
    style(--name: "ManzDev"): indigo;
    style(--name: "CyberManzDev"): red;
    else: grey
  );
}
```

Funciones en CSS

```
@function --gradient(--colors) {  
  --shape: circle 150px;  
  --position: 100% 50%;  
  result: radial-gradient(  
    var(--shape) at var(--position), var(--colors)  
  );  
}  
  
.box {  
  --colors: black, indigo, deeppink;  
  width: 200px;  
  height: 75px;  
  background: --gradient(var(--colors));  
}
```

- En CSS tenemos la regla **@function**
- Se usan como las variables CSS pero con paréntesis
- Las funciones devuelven resultados
- Permiten parámetros
- Hacen más semántico y reutilizable el CSS

Funciones random

- Con cache `random()` / `random-item()`
- Sin cache (usando `per-element`)
- Con cache selectivo (usando `--id`)

```
.container {  
  width: random(100px, 500px);           /* Con cache: */  
  height: random(100px, 500px);          /* Mismo input, mismo valor aleatorio */  
  
  width: random(--id, 100px, 500px);      /* Con cache: Mismo valor aleatorio en mismo --id */  
  width: random(per-element, 100px, 500px); /* Sin cache: Valor aleatorio */  
  
  width: random(100px, 500px, by 50px);   /* Con saltos de 50px en 50px */  
  background: random-item(indigo, deeppink, tomato); /* Elige un valor aleatorio de la lista */  
}
```


Tipos de datos en CSS

- Ejemplo → en regla `@property`
- Ejemplo → en función `attr()`
- Hay muchos tipos de datos

```
@property --x {  
  syntax: "<length>"; /* La variable --x */  
  inherits: true; /* Es de tipo longitud */  
  initial-value: 0; /* Es heredable */  
  /* Y tiene este valor por defecto */  
}  
  
.element {  
  display: inline flow-root;  
  padding: 4px 8px;  
  background: attr(data-color type(<color>));  
  color: white;  
}
```

```
<!-- El color se toma del atributo -->  
<div class="element"  
  data-color="indigo">Manz</div>
```


Transpilar código CSS

```
.element {  
  background: color-mix(...);  
  color: light-dark(...);  
  
  &:is(...) { /* ... */ }  
  
  .child { /* ... */ }  
}
```

```
.element {  
  background: #848833; /* Color procesado y "aplanado"  
  color: var(--light-dark); /* Versión compatible si no  
}  
  
.parent .child-1,  
.parent .child-2,  
.parent .child-3 { /* ... */ } /* Versión compatible s  
  
.parent .child { /* ... */ } /* Sin CSS nesting (nav
```

- [PostCSS](#)
- [LightningCSS](#)

Referencias

- [CheatSheet CSS](#)
- [bootcamp.manz.dev](#)